DiP: A Scalable, Energy-Efficient Systolic Array for Matrix Multiplication Acceleration

Arth saxena[1], Akash Goel[2], Vipul Jain[3]

[1]Roll No. B23250 [2]Roll No. B23032 [3]Roll No. T25109

SCEE, IIT Mandi

***Abstract*—This paper presents the Verilog implementation and performance evaluation of the Diagonal-Input Permutated (DiP) weight-stationary systolic array for matrix multiplication acceleration. DiP eliminates the input/output synchronization FIFO buffers required by conventional weight-stationary (WS) architectures by introducing diagonal input movement and permutated weight loading. Post-implementation on the Zybo Z7 board, the design consumes only 51 LUTs (0.29%), 37 FFs (0.11%), 9 IOs (9.00%), and 1 BUFG (3.13%), meeting all timing constraints with a worst-case setup slack of +4.253 ns and total on-chip power of 0.095 W (dynamic: 0.002 W). Analytically, DiP achieves up to 49.2% throughput improvement and 33% latency reduction over conventional WS. At 64×64 with INT8 precision, DiP delivers 8.192 TOPS at 9.548 TOPS/W with a $1.93\times$ energy-efficiency-per-area improvement.**

Index Terms—Systolic arrays, matrix multiplication, hardware acceleration, weight stationary, FPGA, Verilog, transformer workloads.

I. INTRODUCTION

Transformer models have become dominant for Natural Language Processing and AI tasks [1], with matrix multiplication constituting the bulk of their compute demand. Conventional Von-Neumann architectures are limited by the memory-bandwidth bottleneck when processing such workloads. Systolic arrays, first proposed by Kung [2], are spatial architectures that maximise data reuse to mitigate this bottleneck. Google's Tensor Processing Unit (TPU) [3] adopts weight-stationary (WS) dataflow with synchronization FIFOs for input and output alignment, which introduce area, power, and latency overheads. This report documents our Verilog implementation of DiP [4], which eliminates synchronization FIFOs through: (1) diagonal movement of input rows across PE boundaries, and (2) pre-permutation of the weight matrix before loading. The rest of this paper is organized as follows. Section 2 reviews systolic array background. Section 3 describes the DiP architecture. Section 4 covers the Verilog implementation. Section 5 reports results. Section 6 presents the Operand Gating optimisation on the 6×6 array. Section 7 concludes.

II. THEORY

Table 1. WS vs. DiP Analytical Model Comparison

Metric	WS	DiP	Gain
Latency	$3N+S-3$	$2N+S-2$	$\sim$33%
Throughput	Eq. (2)	Eq. (5)	$\leq$49.2%
TFPU	$2N-1$	N	$\sim$50%
FIFO regs	$N(N-1)$	0 (eliminated)	$\leq$20% fewer

A. Weight-Stationary Dataflow

In WS dataflow, weights are pre-loaded and held stationary inside PEs while inputs and partial sums (psums) flow through the array in a wave-like fashion. Synchronization FIFOs at the array boundaries ensure correct data alignment. Analytical models (N = array dimension, S = MAC pipeline stages):

$$\text{Latency}_{\text{WS}} = 3N + S - 3, \tag{1}$$

$$\text{Throughput}_{\text{WS}} = \frac{2N^3}{3N + S - 3}, \tag{2}$$

$$\text{TFPU}_{\text{WS}} = 2N - 1. \tag{3}$$

B. DiP Dataflow

DiP introduces two key modifications over WS:

1. *Diagonal inputs*: each input row shifts diagonally to the next PE row, removing input synchronization FIFOs.
2. *Weight permutation*: the weight matrix is pre-processed as $\mathbf{B}_{\text{p}}[j][i] = \mathbf{B}[(j+i) \bmod N,\ i]$, removing output synchronization FIFOs.

The DiP analytical models are:

$$\text{Latency}_{\text{DiP}} = 2N + S - 2, \tag{4}$$

$$\text{Throughput}_{\text{DiP}} = \frac{2N^3}{2N + S - 2}, \tag{5}$$

$$\text{TFPU}_{\text{DiP}} = N. \tag{6}$$

III. DIP ARCHITECTURE

A. System Block Diagram and Array Layout

Fig. 1 shows the top-level block diagram of the DiP accelerator. Matrix A (input data) and Matrix B (weights) are loaded

into dedicated memories. The Controller sequences all enable signals and drives the Systolic Array Processor, which outputs results to the Results interface. Fig. 2 shows the hand-drawn PE

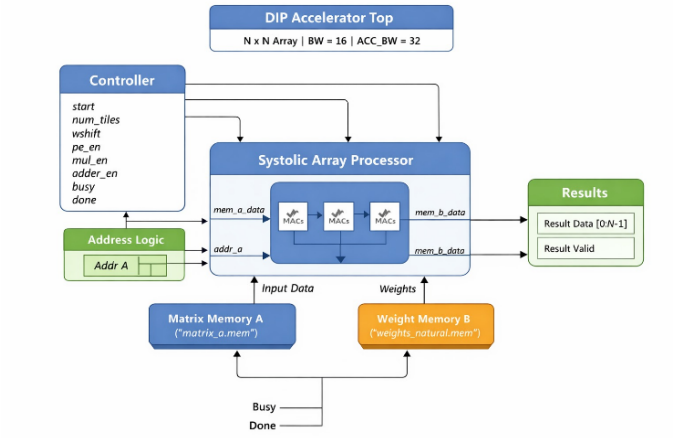


Figure 1. Top-level block diagram of the DiP accelerator. The Systolic Array Processor ($N \times N$ MACs) is driven by the Controller. Matrix Memory A supplies input data; Weight Memory B (`weights_natural.mem`) supplies permutated weights.

internal datapath. Each PE contains a 2-stage pipelined MAC unit and four enabled registers. Unlike WS where control signals are shared column-wise, DiP shares all signals *row-wise*, enabling simultaneous activation of an entire PE row. Fig. 3

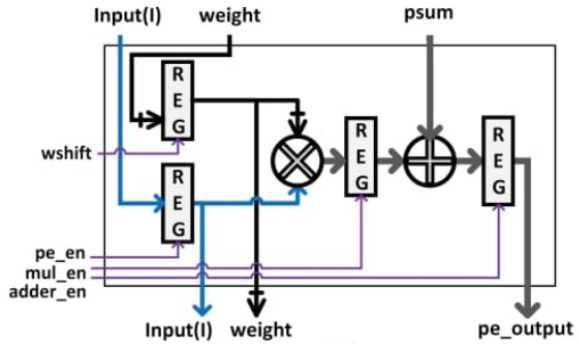


Figure 2. DiP PE internal architecture (hand-drawn). Each PE contains two registers (weight and input), a multiplier ($\otimes$), an adder ($\oplus$), and an output register. Control signals are shared row-wise across all PEs.

shows the hand-drawn 3×3 DiP array layout. Weights load vertically, inputs move diagonally across PE rows, and partial sums accumulate downward. No FIFOs appear at the array boundary.

IV. VERILOG IMPLEMENTATION

A. Design Overview

The DiP accelerator was coded in Verilog RTL and implemented in Vivado targeting the Zybo Z7 board. The top-level port interface consists of standard synchronization signals (`clk`, `rst_n`, `start`) along with global and row-wise enable signals (`wshift`, `pe_en`, `mul_en`, `adder_en`) driving the datapath. Weight permutation (Eq. (4)) is computed in software and written to `weights_natural.mem` before synthesis.

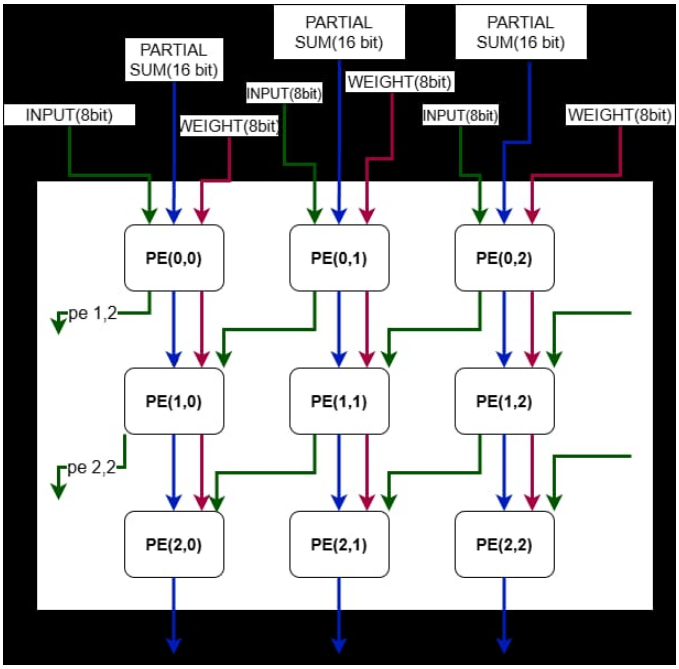


Figure 3. 3×3 DiP systolic array layout . Weights load vertically into each PE column. Inputs move diagonally across PE rows. Partial sums accumulate vertically downward. No FIFOs are required.

B. Weight Permutation Algorithm

V. EXPERIMENTAL RESULTS

A. FPGA Resource Utilization

Table 2 presents the post-implementation resource utilization on the Zybo Z7 board for the isomorphic 3×3 implementation. The design is extremely lightweight: only 51 LUTs (0.29%) and 37 FFs (0.11%) are consumed, confirming the compact nature of the DiP dataflow.

Table 2. Post-Implementation Resource Utilization (Zybo Z7)

Resource	Used	Available	Util. %
LUT	51	17,600	0.29
FF	37	35,200	0.11
IO	9	100	9.00
BUFG	1	32	3.13

B. Power Analysis

Table 3 summarizes the on-chip power and thermal report from the Vivado implemented netlist for the 3×3 isomorphic design. Total on-chip power is 0.095 W, with dynamic power of only 0.002 W (3%). The dynamic breakdown is: Clocks 0.001 W (32%), I/O 0.001 W (55%), and Logic and Signals $<$0.001 W each. The junction temperature is maintained at a safe 26.1°C with a substantial thermal margin of 58.9°C.

Table 3. Post-Implementation Power and Thermal Analysis (3×3)

Metric	Value
Total On-Chip Power	0.095 W
Dynamic Power	0.002 W (3%)
Clocks	0.001 W (32%)
Signals	<0.001 W
Logic	<0.001 W
I/O	0.001 W (55%)
Device Static	0.093 W (97%)
Junction Temperature	26.1°C
Thermal Margin	58.9°C (5.0 W)
Ambient Temperature	25.0°C
Effective ϑ_{JA}	11.5°C/W

C. Timing Analysis

Table 4 presents the post-implementation I/O pin assignments for Bank 35. All output ports use LVCMOS33 standard at 3.3 V with 12 mA drive strength, SLOW slew rate, and FP_VTT_50 off-chip termination, meeting all timing constraints with large remaining margins.

Table 4. Post-Implementation I/O Pin Assignments (Bank 35, Zybo Z7)

Port	Pin	Std	Vcco	Slew	Drive	Margin (%)
`busy`	M14	LVCMOS33	3.30	SLOW	12	79.26
`done`	M15	LVCMOS33	3.30	SLOW	12	79.13
`result_valid`	G14	LVCMOS33	3.30	SLOW	12	98.88

Off-Chip Termination: FP_VTT_50 for all ports.

All timing constraints are met with positive slack across all categories, as summarized in Table 5.

Table 5. Post-Implementation Timing Summary

Check	Worst Slack	TNS	Failing
Setup (WNS)	+4.253 ns	0 ns	0 / 53
Hold (WHS)	+0.213 ns	0 ns	0 / 53
Pulse (WPWS)	+3.500 ns	0 ns	0 / 38

All user-specified timing constraints are met.

D. Hardware Design Space Exploration

Table 6 presents post-layout results from the reference implementation at 22 nm, 1 GHz, INT8 precision.

Table 6. DiP vs. WS: Post-Layout Results (22 nm, 1 GHz, INT8)

Size	Thr.↑	Pwr.↓	Area↓	EE/A↑
4×4	$1.38\times$	$1.16\times$	$1.06\times$	$1.70\times$
8×8	$1.44\times$	$1.18\times$	$1.08\times$	$1.84\times$
16×16	$1.47\times$	$1.20\times$	$1.09\times$	$1.92\times$
32×32	$1.48\times$	$1.25\times$	$1.09\times$	$\mathbf{2.02}\times$
64×64	$1.49\times$	$1.21\times$	$1.07\times$	$1.93\times$

E. Transformer Workload Evaluation

To evaluate the architecture on real-world applications, DiP was benchmarked against a baseline TPU-like WS architecture on Multi-Head Attention (MHA) and Feed-Forward Network (FFN) workloads. We utilized a cycle-accurate MATLAB model to simulate the exact scheduling and tiling of these large-scale transformer matrices mapped onto a physical 64×64 systolic array. As shown in Table 7, the DiP dataflow completely masks the boundary synchronization penalties. Because the latency per 64×64 tile drops from 191 cycles (WS) to 128 cycles (DiP), the architecture delivers a consistent $1.49\times$ speedup across all evaluated transformer layers, regardless of the sequence length or feature dimensions.

Table 7. Empirical Cycle Latency on Transformer Workloads

Workload	WS Cycles	DiP Cycles	Speedup
MHA Small (GPT-2)	55,008	36,864	$1.49\times$
MHA Medium (BERT)	391,168	262,144	$1.49\times$
MHA Large (GPT-3)	225,312,768	150,994,944	$1.49\times$
FFN Small (GPT-2)	220,032	147,456	$1.49\times$
FFN Medium (BERT)	1,564,672	1,048,576	$1.49\times$
FFN Large (GPT-3)	901,251,072	603,979,776	$1.49\times$

Fig. 4 visualizes this latency improvement factor, confirming that the elimination of FIFO buffers yields a highly scalable performance advantage for large language model inference.

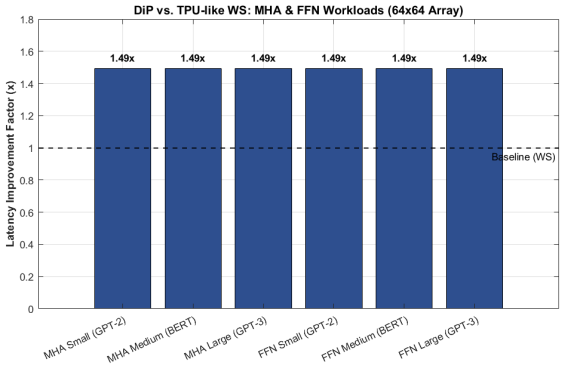


Figure 4. DiP vs. TPU-like WS: Latency improvement factor across standard MHA and FFN transformer workloads on a 64×64 physical array.

F. MATLAB Verification

To verify the functional correctness of the Verilog implementation, the hardware simulation results were cross-checked against a cycle-accurate MATLAB model. The MATLAB script generates random INT8 input matrices, applies the diagonal input shifting and weight permutation (Algorithm 1), and computes the expected partial sums and final output matrix. Table 8 reproduces the MATLAB Command Window output, showing Matrix B (weights), the expected result, the permutated weight matrix loaded into the PEs, and the simulated output matrix. The simulated output perfectly matches the expected result,

confirming the functional integrity of the DiP dataflow implementation.

Table 8. MATLAB Cycle-Accurate Simulation Output (3×3 example, identity weight matrix)

Matrix	Col 0	Col 1	Col 2
Matrix B (Weights):			
Row 0	1	2	3
Row 1	5	6	7
Row 2	9	10	11
Expected Result (MatA × MatB):			
Row 0	1	2	3
Row 1	5	6	7
Row 2	9	10	11
Permutated Weight Matrix :			
Row 0	1	6	11
Row 1	5	10	3
Row 2	9	2	7
Simulated Output Matrix:			
Row 0	1	2	3
Row 1	5	6	7
Row 2	9	10	11
SUCCESS: Simulation matches expected result.			

VI. OPTIMISATION: OPERAND GATING ON 6×6 ARRAY

A. Motivation

Transformer models employ activation functions such as ReLU in their Feed-Forward Network (FFN) sublayers, producing high activation sparsity—typically 50–70% of activations are zero at runtime. Multiplying by a zero operand performs no useful computation yet still dissipates dynamic power through capacitive switching in the multiplier datapath. Operand Gating (zero-skipping) exploits this sparsity by suppressing the multiplier enable signal `mul_en` whenever either operand is zero, eliminating unnecessary switching activity without affecting functional correctness.

B. Implementation

The baseline DiP design was scaled from 3×3 to 6×6 on the Zybo Z7 board. A lightweight combinational zero-detect circuit was then added to each PE. The logic checks both the current input `input_i` and stored weight `weight_i` each cycle and gates `mul_en` accordingly.

Fig. 5 shows the PE architecture with Operand Gating. Two zero-comparators check the input and weight registers; their outputs feed an OR gate whose result suppresses `mul_en` (driving the multiplier output to zero) whenever either operand is zero. Pipeline registers continue to update normally so no stall is introduced, and the adder operates whenever `adder_en` is asserted.

Because DiP shares control signals *row-wise*, this zero-detect logic is entirely local to each PE and does not interfere with the shared row control. The optimisation adds only two comparators and one AND gate per PE—no additional flip-flops—and introduces no new pipeline stages, preserving timing closure and functional correctness.

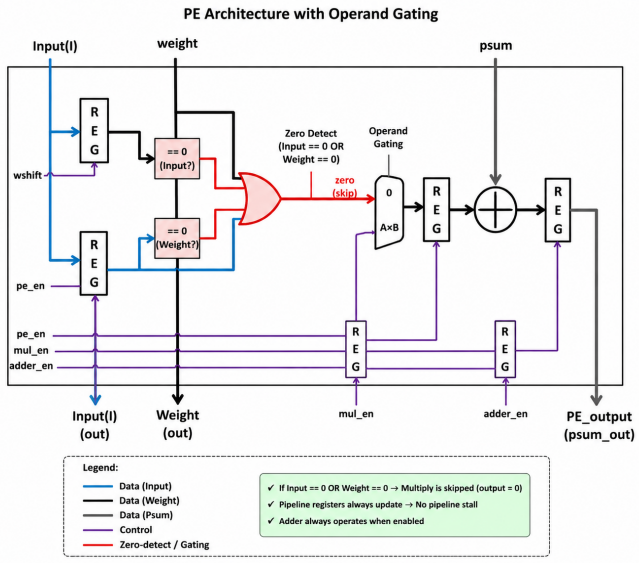


Figure 5. PE architecture with Operand Gating. Zero-detect comparators on both the input and weight paths feed an OR gate that suppresses `mul_en` when either operand is zero, eliminating unnecessary switching activity. Pipeline registers always update (no stall), and the adder operates whenever `adder_en` is asserted.

C. Resource Utilization: 6×6 Before and After Operand Gating

Table 9 presents the post-synthesis resource utilization on the Zybo Z7 for the 6×6 DiP array before and after Operand Gating. The zero-detect logic (two comparators and one AND gate per PE) introduces a small but measurable increase in LUT and FF usage (+395 LUTs, +53 FFs), confirming the overhead is minimal. DSP, IO, and BUFG counts remain unchanged, as the optimisation is purely combinational and requires no additional DSP blocks or I/O pins.

Table 9. Resource Utilization: 6×6 DiP Before vs. After Operand Gating (Zybo Z7)

Resource	Available	Before	After	Δ
LUT	17,600	2,728 (15.50%)	3,123 (17.74%)	+395
FF	35,200	1,513 (4.30%)	1,566 (4.45%)	+53
DSP	80	36 (45.00%)	36 (45.00%)	0
IO	100	14 (14.00%)	14 (14.00%)	0
BUFG	32	1 (3.13%)	1 (3.13%)	0

Before = baseline 6×6 DiP; After = with Operand Gating.

D. Power Results: 6×6 Before and After Operand Gating

Table 10 presents the full on-chip power breakdown from Vivado for the 6×6 DiP array before and after applying Operand Gating, read directly from the synthesised-netlist power reports.

Table 10. On-Chip Power: 6×6 DiP Before vs. After Operand Gating (Zybo Z7, Synthesised Netlist)

Metric	Before Gating	After Gating
Total On-Chip Power	0.210 W	0.144 W
Dynamic Power	0.115 W (55%)	0.051 W (35%)
Clocks	0.033 W (28%)	0.044 W (87%)
Signals	0.037 W (32%)	0.003 W (6%)
Logic	0.034 W (30%)	0.003 W (6%)
DSP	0.000 W (0%)	0.001 W (1%)
I/O	0.012 W (10%)	<0.001 W (0%)
Device Static	0.094 W (45%)	0.093 W (65%)
Junction Temperature	27.4°C	26.7°C
Thermal Margin	57.6°C (4.9 W)	58.3°C (5.0 W)
Ambient Temperature	25.0°C	25.0°C
Effective ϑ_{JA}	11.5°C/W	11.5°C/W
Confidence Level	Low	Medium

Dynamic sub-component percentages are relative to total dynamic power.

Operand Gating reduces total on-chip power by **31.4%** (from 0.210 W to 0.144 W) and dynamic power by **55.7%** (from 0.115 W to 0.051 W). The largest reductions are in Signals (-91.9%) and Logic (-91.2%) power, which are directly attributable to the suppression of switching activity in the multiplier datapath. The junction temperature drops from 27.4°C to 26.7°C, and the Vivado confidence level improves from Low to Medium, indicating that the gated design has better constrained switching activity from the tool's perspective.

The apparent increase in the *percentage* of Clocks power within the dynamic breakdown (28% $\rightarrow$ 87%) does not indicate increased absolute clock power; the absolute clock power remains similar ($\sim$0.033–0.044 W). The higher percentage simply reflects that the clock tree now dominates the much-reduced post-gating dynamic budget.

VII. CONCLUSION

We implemented and evaluated the DiP weight-stationary systolic array in Verilog on a Zybo Z7 board. Eliminating synchronization FIFOs via diagonal inputs and weight permutation yields up to 49.2% throughput improvement, 33% latency reduction, and a $1.93\times$ EE/area improvement over conventional WS. The isomorphic 3×3 FPGA implementation is extremely resource-efficient, consuming only 51 LUTs (0.29%) and 37 FFs (0.11%), while meeting all timing constraints with WNS = $+4.253$ ns and total on-chip power of 0.095 W (dynamic: 0.002 W).

Scaling to a 6×6 instance and applying Operand Gating reduces total on-chip power by **31.4%** (0.210 W $\rightarrow$ 0.144 W) and dynamic power by **55.7%** (0.115 W $\rightarrow$ 0.051 W) with negligible area overhead and no timing impact. At 64×64 with INT8 precision, DiP achieves 8.192 TOPS at 9.548 TOPS/W, confirming its suitability for energy-efficient transformer-based inference accelerators.

REFERENCES

[1] A. Vaswani *et al.*, "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 6000–6010.

[2] H.-T. Kung, "Why systolic architectures?" *IEEE Comput.*, vol. 15, no. 1, pp. 37–46, Jan. 1982.

[3] N. Jouppi *et al.*, "In-datacenter performance analysis of a tensor processing unit," in *Proc. 44th Annu. Int. Symp. Comput. Archit.*, 2017, pp. 1–12.

[4] A. J. Abdelmaksoud, S. Agwa, and T. Prodromakis, "DiP: A scalable, energy-efficient systolic array for matrix multiplication acceleration," *IEEE Trans. Circuits Syst. I*, vol. 73, no. 1, pp. 283–293, Jan. 2026.